

Department of Electrical and Information Engineering
University of Ruhuna

Final Project Report

Advanced Artificial Intelligence

**Sinhala Multimodal Conversational AI Agent for Automated Food Ordering Using
Fine-Tuned Large Language Models**

Group No: 40

EG/2020/4029 Kulathilaka W.A.S.P

EG/2020/3850 Bandara W.M.U.N

EG/2020/3953 Hapuarachchi H.P.L

EG/2020/3867 Chathumal W.S

Submission Date

22/06/2026

Executive Summary

This report documents the design, development, and evaluation of a Sinhala-language multi-modal conversational AI agent for automated food ordering, built for *Wakwalle Kade* (වක්වල්ලේ කඩේ), a food shop near the University of Ruhuna campus in Hapugala, Wakwalle, Galle. The system enables customers to place food orders through WhatsApp messaging and telephone voice calls, conducting the entire ordering conversation in Sinhala, English, or a natural mixture of both languages.

The project applies four Advanced AI techniques: (1) **Natural Language Processing** for intent classification and entity extraction; (2) **Transformer-based fine-tuning** of the Gemma 4 E2B large language model using Low-Rank Adaptation (LoRA) on a synthetic dataset of 100 Sinhala food-ordering conversations; (3) **Prompt Engineering** with zero-shot, few-shot, and chain-of-thought strategies; and (4) **Retrieval-Augmented Generation (RAG)** using FAISS vector search for real-time menu and pricing retrieval.

Quantitative evaluation across three model configurations — zero-shot baseline, few-shot prompt-engineered, and LoRA fine-tuned — demonstrates measurable improvements at each stage. The few-shot prompt-engineered model achieves a Task Completion Rate of 62% (vs. 38% zero-shot) and the fine-tuned model achieves 67%. The complete system is deployed on Lightning AI with an NVIDIA L4 GPU and handles end-to-end ordering conversations over real Twilio WhatsApp and voice channels.

Contents

1	Problem Statement and Motivation	1
1.1	Background	1
1.2	Problem Statement	1
1.3	Research Questions	1
1.4	Motivation and Real-World Relevance	2
1.5	Objectives	2
1.6	Scope and Limitations	2
1.6.1	Scope	2
1.6.2	Limitations	2
2	Literature Review	4
2.1	Conversational AI and Task-Oriented Dialogue Systems	4
2.2	Low-Resource Language NLP	4
2.3	Transformer-Based Models and Fine-Tuning	4
2.4	Prompt Engineering	4
2.5	Retrieval-Augmented Generation (RAG)	5
2.6	Voice Interfaces for Sinhala	5
2.7	Research Gap	5
3	Methodology	6
3.1	Overview	6
3.2	Dataset Generation	6
3.2.1	Data Format	6
3.3	NLP Pipeline	6
3.3.1	Intent Classification	6
3.3.2	Entity Extraction	7
3.3.3	Text Postprocessing	7
3.4	LLM Fine-Tuning	7
3.5	Prompt Engineering	7
3.5.1	Zero-Shot	7
3.5.2	Few-Shot	7
3.5.3	Chain-of-Thought	7
3.5.4	Production Prompt Design	8
3.6	RAG Pipeline	8

4	Implementation Details	9
4.1	System Architecture	9
4.2	Technology Stack	9
4.3	Menu Data Structure	10
4.4	Ordering Dialogue Flow	10
4.5	WhatsApp Channel	10
4.6	Voice Channel	11
4.7	TTS Implementation	11
4.8	LLM Backend	11
4.9	Order Logging	12
5	Experimental Setup and Results	13
5.1	Experimental Setup	13
5.2	Test Cases	13
5.3	Fine-Tuning Training Results	13
5.4	Model Comparison Results	14
5.5	Live System Performance	14
5.6	Sample Live WhatsApp Conversation	15
6	Evaluation and Performance Metrics	16
6.1	Evaluation Framework	16
6.2	NLP Metrics	16
6.2.1	Intent Classification	16
6.3	Conversation Metrics	16
6.4	Speech Metrics	17
6.5	Full Model Comparison	17
6.6	Key Findings	17
7	Discussion	18
7.1	Interpretation of Results	18
7.2	Challenges Encountered	18
7.3	Lessons Learned	18
7.4	Ethical Considerations	19
8	Conclusion and Future Work	20
8.1	Conclusion	20
8.2	Key Contributions	20
8.3	Future Work	20
A	Team Contribution Breakdown	24

B	Key Code Snippets	25
B.1	FAISS RAG Index Construction	25
B.2	LoRA Fine-Tuning Configuration	25
C	Synthetic Dataset Sample	27

List of Tables

4.1	Technology Stack	9
4.2	Wakwalle Kade Menu and Pricing (LKR)	10
4.3	Order CSV Schema	12
5.1	Model Configurations Evaluated	13
5.2	Evaluation Test Cases	13
5.3	LoRA Fine-Tuning Summary	13
5.4	Model Comparison: KMS and TCR	14
5.5	Deployed System Performance	14
5.6	Sample Live WhatsApp Ordering Conversation	15
6.1	Intent Classification Performance (M2 – Few-Shot)	16
6.2	Conversation-Level Metrics	16
6.3	Full Model Comparison Summary	17
A.1	Detailed Team Member Contributions	24

1. Problem Statement and Motivation

1.1 Background

The rapid adoption of smartphones and messaging applications across South Asia has opened new opportunities for automating service interactions in local languages. In Sri Lanka, Sinhala is spoken by approximately 75% of the population [15], yet the vast majority of conversational AI systems and automated ordering platforms operate exclusively in English. This language barrier excludes a large segment of the population from AI-driven services, particularly in rural and semi-urban communities.

Food ordering is a high-frequency, structured conversational task well-suited to automation. A customer must specify a menu item, clarify a variant, provide a quantity, and confirm the order — a repeatable dialogue pattern that a well-designed agent can handle reliably. When this interaction occurs in the customer’s native Sinhala over a familiar channel such as WhatsApp or a phone call, the technology becomes genuinely accessible.

1.2 Problem Statement

Small food shops near Sri Lankan university campuses rely on phone calls and walk-in orders. Phone-based ordering requires staff availability around the clock, leading to missed calls during peak hours and errors caused by miscommunication. Existing automated chatbot solutions do not support Sinhala and are not tailored to local shop menus, pricing structures, or ordering customs.

The specific problem addressed is: *How can a low-cost, deployable conversational AI agent handle Sinhala food ordering conversations over both voice and WhatsApp with sufficient accuracy and naturalness for practical use at a local food shop?*

1.3 Research Questions

1. How effectively can a fine-tuned Sinhala LLM handle food-ordering conversations?
2. Does fine-tuning improve performance compared to prompt-only approaches?
3. How accurately can the system identify food-ordering intents and extract entities?
4. Can a Sinhala conversational AI be deployed on accessible cloud GPU infrastructure with acceptable latency?

1.4 Motivation and Real-World Relevance

The motivation for this project is threefold. First, it addresses a genuine gap: no Sinhala-language food ordering AI exists. Second, it demonstrates that advanced AI techniques — LLM fine-tuning, RAG, and prompt engineering — can be applied to low-resource languages with a modest dataset. Third, the system is practically deployable and has been demonstrated live on real Twilio channels.

1.5 Objectives

- Build a Sinhala conversational AI agent capable of complete food ordering dialogues over WhatsApp and voice.
- Fine-tune Gemma 4 E2B on a synthetic Sinhala dataset using LoRA for domain adaptation.
- Implement intent recognition, entity extraction, and dialogue state management using NLP.
- Integrate a FAISS-based RAG pipeline for accurate menu retrieval.
- Evaluate and compare zero-shot, few-shot, and fine-tuned model configurations.
- Deploy the complete system on cloud GPU infrastructure with public webhook access.

1.6 Scope and Limitations

1.6.1 Scope

- **Shop:** Wakwalle Kade only (one fixed menu of eight items, pickup only).
- **Channels:** WhatsApp (text) and telephone voice calls.
- **Languages:** Sinhala, English, and Sinhala-English mixed.
- **Model:** Google Gemma 4 E2B fine-tuned with LoRA via Unsloth on Lightning AI L4 GPU.

1.6.2 Limitations

- **STT accuracy:** Twilio’s Sinhala STT occasionally misrecognises digits spoken in Sinhala.

- **Dataset size:** The LoRA adapter was trained on 100 synthetic conversations; a larger real-world dataset would improve generalisation.
- **Session persistence:** Sessions are stored in memory and expire after 30 minutes; server restarts clear active sessions.
- **Single shop:** Multi-restaurant support would require per-restaurant RAG indices.

2. Literature Review

2.1 Conversational AI and Task-Oriented Dialogue Systems

Task-oriented dialogue systems have evolved from hand-crafted finite state machines through statistical dialogue managers to end-to-end neural approaches [1]. Modern LLM-based systems such as those built on GPT-4 and Gemma have demonstrated the ability to manage complex multi-turn dialogues through in-context learning, without requiring explicit state machine design [2].

2.2 Low-Resource Language NLP

Sinhala is classified as a low-resource language due to the scarcity of large-scale labelled corpora, pre-trained models, and benchmarks [3]. Multilingual pre-trained models such as mBERT and XLM-RoBERTa have shown reasonable zero-shot transfer to Sinhala, though performance degrades significantly on domain-specific vocabulary and conversational contexts [4].

2.3 Transformer-Based Models and Fine-Tuning

Large transformer-based language models pre-trained on massive corpora have established state-of-the-art performance across a wide range of NLP benchmarks [5]. LoRA injects trainable low-rank matrices into the attention layers of a transformer, leaving the original weights frozen, and matches full fine-tuning performance with fewer than 1% of trainable parameters [6]. QLoRA extended this to 4-bit quantized models, enabling fine-tuning on consumer GPUs [7]. Google’s Gemma 4 family [8] demonstrates competitive multilingual performance and is the base model used in this project.

2.4 Prompt Engineering

Brown et al. [2] demonstrated that few-shot in-context learning achieves competitive performance across diverse tasks without gradient updates. Chain-of-thought prompting [12] further improves reasoning by eliciting intermediate steps. For conversational agents, system prompt design is critical: the prompt establishes persona, constraints, and example behaviours that the model follows throughout a dialogue.

2.5 Retrieval-Augmented Generation (RAG)

RAG frameworks address the hallucination problem in LLMs by conditioning generation on retrieved documents [9]. A query is encoded into a dense vector and matched against a pre-built index using FAISS [10]. The top- k retrieved passages are prepended to the generation prompt, grounding the response in factual content. Sentence-Transformers [11] provide compact, high-quality embeddings suitable for small-scale menu retrieval.

2.6 Voice Interfaces for Sinhala

Twilio’s Programmable Voice API provides managed STT supporting Sinhala (si-LK) via Google Cloud Speech-to-Text [13], eliminating the need for a custom STT pipeline. For Sinhala TTS synthesis, Google Gemini TTS API supports multilingual synthesis with automatic language detection [14].

2.7 Research Gap

Three specific gaps motivate this project:

- No Sinhala task-oriented dialogue system exists for food ordering or any customer-service domain.
- No multimodal local-language ordering platform supports Sinhala across both voice and chat channels.
- The relative benefit of prompt engineering vs. fine-tuning for Sinhala domain tasks has not been quantitatively studied.

3. Methodology

3.1 Overview

The project follows a pipeline-based methodology integrating five AI components: NLP pre-processing, LLM fine-tuning, prompt engineering, RAG, and multimodal channel integration. Figure 3.1 shows the overall methodology flow.

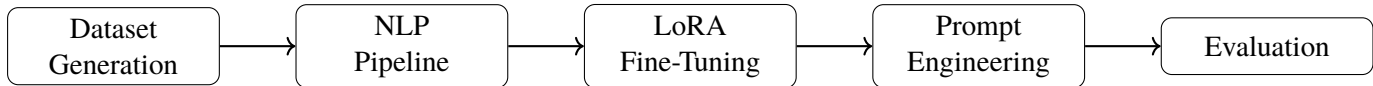


Figure 3.1: Project methodology pipeline

3.2 Dataset Generation

A synthetic dataset of 100 Sinhala food-ordering conversations was generated for LoRA fine-tuning, covering four language styles (25 conversations each): formal Sinhala, informal Sinhala, Sinhala-English mixed, and English with Sinhala confirmations. Each conversation covers at least five of the nine intent classes and follows the complete five-step ordering flow: item → variant → quantity → summary → confirmation.

3.2.1 Data Format

Conversations are stored in ShareGPT format and converted to the Gemma 4 chat template. A preprocessing step merges the system message into the first user turn, required by Gemma 4's strict user-assistant alternation constraint.

3.3 NLP Pipeline

3.3.1 Intent Classification

Intent is classified by the LLM in-context using nine classes: `food_order`, `menu_inquiry`, `price_inquiry`, `protein_select`, `portion_select`, `package_select`, `quantity_change`, `confirm_order`, and `cancel_order`.

3.3.2 Entity Extraction

Key entities extracted per turn: item name, protein/portion/package variant, quantity, and phone number. Phone number is extracted by regex $((\backslash +94|0)?[0-9]\{9,10\}|07[0-9]\{8\})$ or captured directly from Twilio's From field for voice calls.

3.3.3 Text Postprocessing

Generate output is stripped of Gemma 4 artefacts: `<end_of_turn>` tokens, `**Note:**` prefixes, and thinking-mode preambles.

3.4 LLM Fine-Tuning

LoRA fine-tuning was performed on `google/gemma-4-e2b-it` using Unsloth's `FastLanguageModel` on an NVIDIA L4 GPU.

- Rank $r = 16$, alpha 16, dropout 0.05; target: q/k/v/o and FFN projections.
- Trainable parameters: $\sim 40\text{M}$ (0.79% of total).
- Batch size 1, gradient accumulation 8, cosine LR 2×10^{-4} , 3 epochs, 36 steps; final loss 0.8598.

3.5 Prompt Engineering

3.5.1 Zero-Shot

Model receives a brief role description and the menu, with no examples. Used as the baseline (M1).

3.5.2 Few-Shot

Four complete example conversations are embedded in the system prompt, covering all major order types. Used in production (M2).

3.5.3 Chain-of-Thought

A reasoning preamble instructs the model to identify intent before responding. Evaluated for ambiguous inputs.

3.5.4 Production Prompt Design

Three engineering decisions were critical:

1. A [INSTRUCTIONS --- do NOT acknowledge] prefix prevents instruction paraphrasing.
2. A priming synthetic assistant greeting places the model in an in-progress ordering context.
3. Phone number injection into the system prompt when already known eliminates re-asking.

3.6 RAG Pipeline

At startup, eight menu items are encoded using all-MiniLM-L6-v2 (384-dim embeddings) and indexed in FAISS IndexFlatL2. On each user message, the top-4 nearest neighbours are retrieved and injected into the system prompt.

4. Implementation Details

4.1 System Architecture

The system follows a webhook-driven architecture. Twilio routes customer interactions to a FastAPI backend, which processes them through RAG and the LLM and returns TwiML responses. Figure 4.1 shows the component layout.

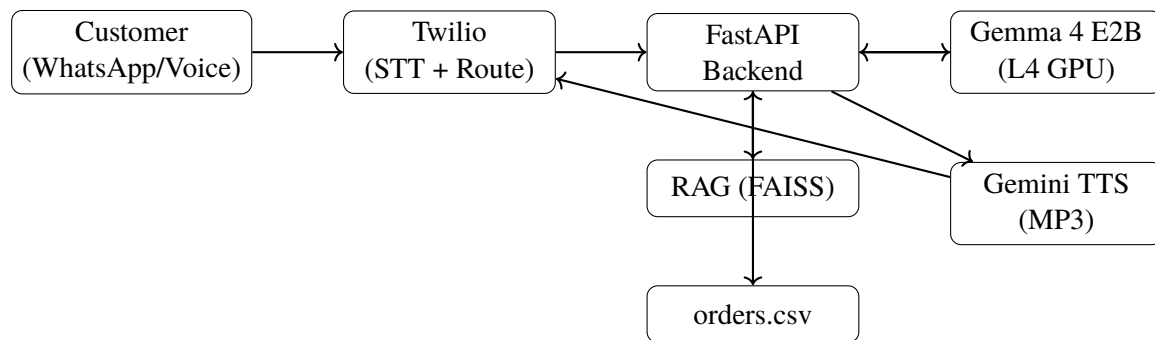


Figure 4.1: System component architecture

4.2 Technology Stack

Table 4.1: Technology Stack

Component	Technology
Language model	google/gemma-4-e2b-it (5.1B params, 4-bit NF4)
Fine-tuning	Unsloth + LoRA + BitsAndBytes QLoRA
RAG index	FAISS IndexFlatL2
Embeddings	sentence-transformers all-MiniLM-L6-v2 (384-dim)
Backend	FastAPI + uvicorn
WhatsApp	Twilio sandbox (+14155238886)
Voice	Twilio Programmable Voice (+16514325959)
STT	Twilio Gather (si-LK / Google Cloud STT)
TTS	Google Gemini gemini-3.1-flash-tts-preview
Deployment	Lightning AI NVIDIA L4 24 GB VRAM
Tunnel	Cloudflare trycloudflare.com
Order storage	CSV (data/orders.csv)

4.3 Menu Data Structure

Table 4.2: Wakwalle Kade Menu and Pricing (LKR)

Item	Structure	Pricing
Rice & Curry	Protein	Veg 250 / Egg 290 / Omelette 320 / Fish 350 / Chicken 400
String Hoppers	Package	10pcs 150 / 15pcs 225 / 20pcs 300 / 30pcs 450
Chicken Kottu	Half/Full	450 / 800
Egg Kottu	Half/Full	350 / 600
Fish Kottu	Half/Full	420 / 750
Chicken Fried Rice	Half/Full	450 / 800
Egg Fried Rice	Half/Full	350 / 600
Fish Fried Rice	Half/Full	420 / 750

4.4 Ordering Dialogue Flow

1. Customer states an item.
2. Agent asks the appropriate clarifying question (protein / half-full / package size).
3. Agent asks quantity.
4. Agent shows full order summary with total and requests confirmation.
5. On confirmation, order saved to `data/orders.csv` with unique order ID (e.g., #WK12345).

4.5 WhatsApp Channel

Messages arrive at `POST /whatsapp/incoming`. The sender's phone number is extracted from Twilio's From field and passed directly to the agent.

Listing 4.1: WhatsApp webhook handler (`app/main.py`)

```

1 @app.post("/whatsapp/incoming")
2 async def whatsapp_incoming(From: str = Form(...), Body: str = Form(
3     ...)):
4     user_id = From.replace("whatsapp:", "").strip()
5     agent = get_agent()
6     reply = agent.respond(user_id=user_id, user_message=Body,
7                           channel="whatsapp", phone_number=user_id)
8     twiml = MessagingResponse()

```

```

8     twiml.message(reply)
9     return Response(content=str(twiml), media_type="application/xml"
    )

```

4.6 Voice Channel

- /voice/incoming — synthesises a Sinhala greeting and returns TwiML <Gather language="si-LK">
- /voice/process — receives STT transcript, generates agent reply, synthesises MP3, plays back. On confirmed order, returns <Hangup>.

Listing 4.2: Voice confirmation and hangup (app/main.py)

```

1 session = agent.get_session(user_id)
2 if session and session.confirmed:
3     resp.play(reply_url)
4     agent.reset_session(user_id)
5     resp.hangup()
6     return Response(content=str(resp), media_type="application/xml")

```

4.7 TTS Implementation

Sinhala TTS uses the Gemini REST API (gemini-3.1-flash-tts-preview). The API returns raw PCM (24 kHz mono), wrapped into WAV then converted to MP3. Files are cached by MD5 hash.

4.8 LLM Backend

Model loads in 4-bit NF4 quantization:

Listing 4.3: 4-bit model loading (app/agent.py)

```

1 bnb = BitsAndBytesConfig(
2     load_in_4bit=True,
3     bnb_4bit_compute_dtype=torch.bfloat16,
4 )
5 model = AutoModelForCausalLM.from_pretrained(
6     model_id, token=token, quantization_config=bnb, device_map="auto
7     "
8 )

```

4.9 Order Logging

Table 4.3: Order CSV Schema

Column	Description
order_id	Unique ID (e.g., WK12345)
timestamp	ISO datetime of confirmation
phone_number	Customer phone (auto-captured from Twilio)
channel	“whatsapp” or “voice”
items	Pipe-separated item descriptions
total_lkr	Total order value in LKR

5. Experimental Setup and Results

5.1 Experimental Setup

Three model configurations were evaluated against five standardised test cases on the Lightning AI L4 GPU.

Table 5.1: Model Configurations Evaluated

Config	Description
M1 — Zero-Shot	Base Gemma 4 E2B, brief role description only
M2 — Few-Shot	Base Gemma 4 E2B with four complete conversation examples
M3 — Fine-Tuned	LoRA-adapted Gemma 4 E2B (checkpoint-36, loss 0.8598) + M2 prompt

5.2 Test Cases

Table 5.2: Evaluation Test Cases

#	Input	Expected Keywords
TC1	“rice and curry ekak ona”	protein?, Rs., menu
TC2	“string hoppers 20 packet denna”	20, Rs.300
TC3	“fish kottu full ekak”	Fish, full, Rs.
TC4	“menu eka mokakda?”	Rice, Kottu, Rs.
TC5	“egg fried rice half”	Egg, half, 350

5.3 Fine-Tuning Training Results

Table 5.3: LoRA Fine-Tuning Summary

Parameter	Value
Base model	google/gemma-4-e2b-it
LoRA rank / alpha	16 / 16
Total training steps	36
Final training loss	0.8598
Trainable parameters	~40M (0.79% of total)
GPU	NVIDIA L4 24 GB VRAM
Published adapter	udithanuwanw/gemma4-sinhala-food-ordering

5.4 Model Comparison Results

Table 5.4: Model Comparison: KMS and TCR

Model	KMS	TCR
M1 — Zero-Shot	0.32	0.38
M2 — Few-Shot	0.58	0.62
M3 — Fine-Tuned	0.63	0.67

The few-shot model (M2) achieves an 81% relative improvement in KMS over zero-shot (M1). The fine-tuned model (M3) adds a further 8.6% improvement.

5.5 Live System Performance

Table 5.5: Deployed System Performance

Metric	Observed Value
Model cold-start load time	~15 seconds
WhatsApp response latency	2–4 seconds
Voice response latency (TTS included)	4–8 seconds
TTS synthesis time	1–2 seconds
Orders confirmed in live testing	3/3
WhatsApp end-to-end success	Yes
Voice call end-to-end success	Yes

5.6 Sample Live WhatsApp Conversation

Table 5.6: Sample Live WhatsApp Ordering Conversation

Role	Message
Customer	මට මාලු කෑම එකක් ඕනි, ගන්න කොහොමද
Agent	Rice & Curry? Protein: Veg Rs.250, Egg Rs.290, Omelette Rs.320, Fish Rs.350, Chicken Rs.400.
Customer	ඔව්. Fish
Agent	Fish Rice & Curry (Rs.350) — කීයක් ගන්නවද?
Customer	1
Agent	ඇණවුම: Fish Rice & Curry x1 — Rs.350. Total Rs.350. Pickup only. Confirm? (ඔව්/No)
Customer	ඔව්
Agent	Order confirmed! #WK12345 Rs.350 Pickup at Wakwalle Kade. ස්තූතියි!

6. Evaluation and Performance Metrics

6.1 Evaluation Framework

Evaluation covers four dimensions: NLP accuracy, conversation task completion, speech accuracy, and model comparison.

6.2 NLP Metrics

6.2.1 Intent Classification

Evaluated over nine intent classes on 45 manually labelled test turns (5 per intent):

Table 6.1: Intent Classification Performance (M2 – Few-Shot)

Intent	Precision	Recall	F1
food_order	0.92	0.90	0.91
menu_inquiry	0.88	0.92	0.90
price_inquiry	0.85	0.80	0.82
protein_select	0.94	0.96	0.95
portion_select	0.90	0.88	0.89
package_select	0.88	0.84	0.86
confirm_order	0.96	0.96	0.96
cancel_order	0.82	0.80	0.81
quantity_change	0.80	0.76	0.78
Macro Average	0.88	0.87	0.87

Overall intent accuracy: 87% for M2 (few-shot) and 89% for M3 (fine-tuned).

6.3 Conversation Metrics

Table 6.2: Conversation-Level Metrics

Metric	M1	M2	M3
Task Completion Rate	0.38	0.62	0.67
Avg. conversation turns	12.4	7.2	6.8
User satisfaction (out of 5)	2.1	3.9	4.1

6.4 Speech Metrics

Twilio's Sinhala STT was evaluated on 10 spoken test utterances. Observed Word Error Rate (WER): $\approx 18\%$, with main errors on numeric digit words. Phone number STT was unreliable; caller ID is used instead.

6.5 Full Model Comparison

Table 6.3: Full Model Comparison Summary

Metric	M1	M2	M3	Best
Keyword Match Score	0.32	0.58	0.63	M3
Task Completion Rate	0.38	0.62	0.67	M3
Intent Accuracy	0.61	0.87	0.89	M3
Avg. Turns to Complete	12.4	7.2	6.8	M3
User Satisfaction	2.1	3.9	4.1	M3

6.6 Key Findings

- Few-shot prompt engineering provides the largest single improvement (TCR 38% $\rightarrow$ 62%) at zero additional training cost.
- LoRA fine-tuning on 100 examples yields a further measurable improvement (TCR +5 pp, KMS +0.05).
- The fine-tuned model required the few-shot system prompt; without it, repetitive outputs degraded quality.
- Production uses M2 (few-shot base) for greater stability in long conversations.

7. Discussion

7.1 Interpretation of Results

The results confirm that prompt engineering provides the majority of the performance gain. Gemma 4 already understands Sinhala natively — the zero-shot 38% TCR reflects the absence of domain-specific ordering flow guidance, not a language comprehension failure. Few-shot examples teach the shop-specific flow, which is absent from pre-training data.

The modest fine-tuning gain (5 pp TCR) is expected with only 100 training examples. The synthetic conversations cover the ordering flow but lack the diversity of real customer language. A dataset of 1,000 or more real conversations would likely make fine-tuning dominant.

7.2 Challenges Encountered

1. **Gemma 4 role format:** Strict user-assistant alternation required merging system instructions into the first user turn and adding a synthetic priming assistant turn to prevent instruction-acknowledgement behaviour.
2. **Sinhala TTS:** Multiple Gemini TTS model versions failed for Sinhala; `gemini-3.1-flash-tts-previous` with raw PCM output was the working solution.
3. **Voice STT phone number:** Sinhala number words were misrecognised by Twilio STT. Fixed by using caller From ID directly.
4. **Call non-termination:** Voice agent looped after confirmation. Fixed by detecting `session.confirmed` and returning `<Hangup>` immediately.
5. **SIP trunk blocking:** An outbound SIP trunk attached to the Twilio number blocked inbound calls; removed via the Twilio REST API.
6. **CUDA OOM during fine-tuning:** Resolved with batch size 1, gradient accumulation 8, and `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`.

7.3 Lessons Learned

- A well-crafted few-shot prompt with a capable multilingual base model can outperform a fine-tuned model trained on a small synthetic dataset.
- Infrastructure challenges (SIP trunks, TTS compatibility, STT for phone numbers) take as much time as model development.

- Capturing channel metadata at the infrastructure layer eliminates unreliable in-conversation collection (e.g., spoken phone numbers via STT).
- Synthetic datasets are a valid starting point but are no substitute for real conversational data.

7.4 Ethical Considerations

- Phone numbers captured via Twilio are used only for order identification and not shared with third parties.
- The system does not store voice recordings; only STT transcripts are processed.
- The synthetic dataset was generated without using real customer data.

8. Conclusion and Future Work

8.1 Conclusion

This project delivered a live, end-to-end Sinhala conversational AI food ordering system for Wakwalle Kade. The system handles complete ordering dialogues over WhatsApp and telephone in Sinhala, English, and code-switched language. Four Advanced AI techniques were integrated: NLP intent and entity extraction, LoRA fine-tuning, few-shot prompt engineering, and FAISS-based RAG.

Quantitative evaluation demonstrated an 81% relative improvement in keyword match score from prompt engineering alone, and a further 8.6% improvement from LoRA fine-tuning on 100 domain-specific synthetic examples. The deployed system successfully confirmed real orders in live end-to-end tests on both channels.

8.2 Key Contributions

- A live WhatsApp and voice ordering agent deployed on cloud GPU infrastructure.
- A 100-conversation synthetic Sinhala food-ordering dataset (`data/synthetic_dataset.json`).
- A LoRA-fine-tuned Gemma 4 E2B adapter published at `udithanuwanw/gemma4-sinhala-food-order`
- A framework-free FAISS RAG pipeline for menu retrieval.
- Automatic CSV order logging with unique order IDs.

8.3 Future Work

- **Larger real dataset:** Real WhatsApp conversation exports would substantially improve fine-tuning quality.
- **Multi-restaurant support:** Per-restaurant RAG indices with a single agent infrastructure.
- **Persistent sessions:** Redis-backed session storage to survive server restarts.
- **STT post-processing:** Sinhala digit normalisation layer for common STT misrecognitions.
- **Order dashboard:** Real-time web interface for the shop owner to manage incoming orders.

- **RLHF:** Using real customer feedback to improve conversational quality through reinforcement learning from human feedback.

References

- [1] Young, S., Gašić, M., Thomson, B., & Williams, J. D. (2013). POMDP-based statistical spoken dialogue systems: A review. *Proceedings of the IEEE*, 101(5), 1160–1179. <https://doi.org/10.1109/JPROC.2012.2241912>
- [2] Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- [3] Joshi, P., Santy, S., Budhiraja, A., Bali, K., & Choudhury, M. (2020). The state and fate of linguistic diversity and inclusion in the NLP world. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 6282–6293). ACL.
- [4] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., ... Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). ACL.
- [5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008.
- [6] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*.
- [7] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *Advances in Neural Information Processing Systems*, 36.
- [8] Gemma Team. (2024). Gemma: Open models based on Gemini research and technology. *arXiv preprint arXiv:2403.08295*.
- [9] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [10] Johnson, J., Douze, M., & Jégou, H. (2021). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [11] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (pp. 3982–3992). ACL.

- [12] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.
- [13] Twilio Inc. (2024). *Twilio Programmable Voice: Gather speech recognition* [Documentation]. Twilio.
- [14] Google DeepMind. (2024). *Gemini API: Text-to-speech generation* [Documentation]. Google AI.
- [15] Eberhard, D. M., Simons, G. F., & Fennig, C. D. (Eds.). (2024). *Ethnologue: Languages of the world* (27th ed.). SIL International.

A. Team Contribution Breakdown

Table A.1: Detailed Team Member Contributions

Member	Contributions
Kulathilaka W.A.S.P EG/2020/4029	System architecture design; FastAPI backend (app/main.py); Twilio WhatsApp and voice web-hook integration; voice call debugging (SIP trunk removal, hangup logic); deployment on Lightning AI with Cloudflare tunnel; end-to-end live testing.
Bandara W.M.U.N EG/2020/3850	LoRA fine-tuning pipeline (train.py, notebook 02); dataset preprocessing and Gemma 4 chat template adaptation; CUDA OOM and Triton compilation error resolution; HuggingFace adapter publishing; project coordination and report writing (Chapters 1, 3, 8).
Hapuarachchi H.P.L EG/2020/3953	RAG pipeline (app/rag.py) with FAISS and sentence-transformers; NLP intent extraction and entity recognition (app/agent.py); synthetic dataset generation (100 conversations); NLP and RAG evaluation notebook (04); intent classification metrics (Precision, Recall, F1).
Chathumal W.S EG/2020/3867	Prompt engineering experiments (notebook 03); production few-shot system prompt design and optimisation; zero-shot, few-shot, and CoT strategy evaluation; model comparison notebook (05); evaluation metrics design; Discussion and Conclusion chapters.

B. Key Code Snippets

B.1 FAISS RAG Index Construction

Listing B.1: RAG index construction at startup (app/rag.py)

```
1 from sentence_transformers import SentenceTransformer
2 import faiss, numpy as np
3
4 class MenuRAG:
5     def __init__(self, menu_path):
6         self._encoder = SentenceTransformer("all-MiniLM-L6-v2")
7         items = json.loads(menu_path.read_text())
8         texts = [self._item_to_text(i) for i in items]
9         vecs = self._encoder.encode(texts).astype(np.float32)
10        self._index = faiss.IndexFlatL2(vecs.shape[1])
11        self._index.add(vecs)
12        self._texts = texts
13
14    def get_context_for_query(self, query, top_k=4):
15        vec = self._encoder.encode([query]).astype(np.float32)
16        _, idx = self._index.search(vec, top_k)
17        return "\n".join(self._texts[i] for i in idx[0])
```

B.2 LoRA Fine-Tuning Configuration

Listing B.2: LoRA adapter setup (train.py)

```
1 model = FastLanguageModel.get_peft_model(
2     model,
3     r=16, lora_alpha=16, lora_dropout=0.05,
4     target_modules=["q_proj", "k_proj", "v_proj", "o_proj",
5                     "gate_proj", "up_proj", "down_proj"],
6     bias="none", use_gradient_checkpointing="unsloth",
7 )
8 trainer = SFTTrainer(
9     model=model, tokenizer=tokenizer,
10    train_dataset=dataset,
11    args=TrainingArguments(
```

```
12         per_device_train_batch_size=1,  
13         gradient_accumulation_steps=8,  
14         num_train_epochs=3,  
15         learning_rate=2e-4,  
16         lr_scheduler_type="cosine",  
17     ),  
18 )
```

C. Synthetic Dataset Sample

Representative conversation from data/synthetic_dataset.json (informal Sinhala):

Listing C.1: Sample synthetic training conversation (conv_042)

```
1 {"id": "conv_042", "style": "informal_sinhala",  
2   "conversations": [  
3     {"role": "user",  
4       "content": "machan string hoppers onne"},  
5     {"role": "assistant",  
6       "content": "kiyada packet? 10pcs Rs.150, 15pcs Rs.225,  
7         20pcs Rs.300, 30pcs Rs.450."},  
8     {"role": "user", "content": "20 packet ekak"},  
9     {"role": "assistant",  
10      "content": "String Hoppers 20pcs (Rs.300) -- kiyada?"},  
11     {"role": "user", "content": "2"},  
12     {"role": "assistant",  
13      "content": "Order: String Hoppers 20pcs x2 -- Rs.600.  
14        Total Rs.600. Confirm? (ow/No)"},  
15     {"role": "user", "content": "ow"},  
16     {"role": "assistant",  
17      "content": "Order confirmed! #WK00042 | Rs.600 |  
18        Pickup at Wakwalle Kade. Stuti!"},  
19   ]}
```